

Rayce Ramsay (1009734888)

Terry Tian (1007663468)

Grant Hamblin (1009860447)

Hardik Shah (1009453690)

CSC490 Assignment A5 - Load testing your application - 10% of Final Grade

Due: Nov 12th, 11pm with a Github PR + Quercus Submission

The goal of this assignment is to find the performance limitations of your project and optimize for them at the code and infrastructure level.

1 Part One - Profiling Execution time (20 marks)

Profile 5 important functions in your codebase using <https://docs.python.org/3/library/profile.html>. Analyze your functions and find ways to improve them. Create a stacked PR called "a5-part1" with your profiling code and optimizations.

- **predict()** in `src/application/backend/app/routers/predictions.py`

Profiling shows that `predict()` spends almost all of its time loading the model on each call (around 80% in `get_model_and_tokenizer`), with the remainder spent on actual generation. This means `predict()` is I/O and initialization bound, not CPU bound on our own code. The clear optimization is to load the model once and reuse it across requests, instead of reloading on every prediction.

- **build_prompt()** in `app/services/prediction_service.py`

This function is pretty cheap however profiling shows that 95% of the time spent is in the parsing of two dates and then subtracting them. To optimize this we will rewrite `parse_date` to use `date.fromisoformat` for standard "YYYY-MM-DD" strings, falling back to `strptime` only when necessary.

- **build_ddi_reference** in `data-pipelines/pipeline.py`

Profiling revealed that `build_ddi_reference()` was by far the largest bottleneck in the entire data pipeline, taking ~110 seconds, with ~83% of the time spent inside a single `groupby().agg(...)` call. The root cause was that pandas invoked our Python-level helper `uniq_list` more than 1.3 million times, causing massive overhead from repeated `dropna()`, list-building, masking, and Python iteration inside the aggregation loop. To optimize this, we replaced `uniq_list` and `first_non_null` with lightweight NumPy-based implementations (`_uniq_nonempty` and `_first_non_null`) that operate directly on the underlying array. We also removed the slow `apply(axis=1)` when constructing pair keys by vectorizing the tuple construction with NumPy. These changes reduced the runtime from ~110s to ~45s (a ~2.4× speedup), significantly speeding up the DDI reference-building stage.

- **extract_severity()** in `app/services/prediction_service.py`

Previously, `extract_severity()` compiled regex patterns and scanned the string on every call. Profiling showed that the majority of time in this function came from repeated regex compilation and case conversions. We moved the patterns to module level precompiled regex objects to reduce these repeated computations.

- **build_patient_ae_tables()** in `data-pipelines/pipeline.py` -

Profiling before optimization showed that `build_patient_ae_tables()` spent most of its time inside a

Python-level `groupby.apply(lambda ...)` operation used to construct per-patient comorbidity lists. While the absolute runtime was already small (~0.067 seconds), this pattern doesn't necessarily scale well as every group triggers Python set operations, repeated `isna()` checks, and per-row iteration. The optimized implementation exhibits a slightly higher overhead on the small Synthea cohort (0.067 -> 0.08s), which is expected as pandas vectorized work incurs a fixed cost. However, the new approach eliminates Python loops and becomes significantly faster as dataset size increases asymptotically, so theoretically this still should optimize things.

2 Part Two: Breaking your application (50 marks)

Design and implement a load testing strategy for your application. Use an open source tool like Locust (<https://locust.io/>) to create load tests for your application. Identify key weakpoints for APIs, dataset sizes, model predictions, etc. One useful strategy is for half your team to take the position of a Red-team (trying to break the application) and the other half to be a Blue-team (trying to defend the application). Create a report with:

- Your load testing strategy
- The weak points you identified in terms of performance
- The opportunities to DOS https://en.wikipedia.org/wiki/Denial-of-service_attack your application
- A link to a video recording of you breaking one portion of your application and how you fixed it with a code walkthrough

Create a stacked PR called "a5-part2" with your load testing code and optimizations.

Load Testing Strategy

Our load testing strategy utilizes Locust to simulate user behavior patterns across both authenticated and unauthenticated traffic flows. We implemented two distinct user classes: representing authenticated users who perform registration, authentication, profile access, and DDI predictions, and unauthenticated users simulating public traffic attempting to access health checks and protected endpoints. Endpoints tested include

- `/health` (Weight 5)
- `/get_authenticated_user` (Protected, Weight 2)
- `/predict_ddi` (Protected, Weight 2)

Testing strategy involved first simulating workloads on a local server, ensuring Locust is able to properly track result metrics. We first simulated a single user workflow (low usage) followed by average expected usage (10 users) and high usage (100 users). After evaluating feedback on local tests, an initial set of fixes were implemented to address shortcomings. Next, we tested against the ALB endpoint and verified the fixes held up.

Identified Weak Points

Identified weaknesses include the following:

- `/predict/` endpoint is a bottleneck since failure to use `asyncio.run_in_executor()` results in application stall.
- High Bedrock latency (several seconds per request) creates backpressure and reduces throughput across ECS Fargate instances.
- No caching layer, causing every request to hit the database or Bedrock API even for repeated or identical operations.
- Hardware constraints result in lack of compute and memory to handle high workloads

DOS Opportunities

The application is vulnerable to DOS attacks due to the absence of rate limiting on any endpoint. An attacker could spam the `/users/register` endpoint to exhaust database connections and

storage, as there are no constraints on frequency. Moreover the /predict/ endpoint represents a severe attack surface since each request triggers an expensive AWS Bedrock API call that can cost money time to complete. Attackers with valid credentials could drain AWS budget and create a backlog that degrades service for legitimate users. A third attack vector is the /login endpoint which has no brute-force protection. There currently is no failed-login tracking or account lockout mechanism which opens the door to credential stuffing attacks.

However, ALB's have built in DOS protection capabilities documented as [Resource level DDoS protection](#). This is automatically enabled for any ALB and rate-limits traffic from known malicious sources. Amazon provides the necessary support "To identify known malicious sources" by using "performing on-host filtering of both direct client IP addresses and X-Forwarded-For (XFF) headers." Additionally AWS WAF and AWS Shield can be leveraged to achieve protection against (D)Dos. At an additional cost, Shield provides automatic network- and transport-layer protection absorbing volumetric attacks before they reach your infrastructure, while WAF adds application-layer defenses by filtering, and rate-limiting incoming requests to block malicious patterns that could overwhelm your API or ALB.

AWS Shield provides automatic network- and transport-layer protection (Layer 3/4), absorbing volumetric attacks before they reach your infrastructure, while **AWS WAF** adds application-layer defenses (Layer 7) by filtering, inspecting, and rate-limiting incoming requests to block malicious patterns that could overwhelm your API or ALB.

Break + Fix

[Link to Video](#)

3 Part Three: Future Scaling Concerns (30 marks)

In this course we discussed how to scale applications but also the importance of right-sizing your scaling approach. Now that you have load tested your application, create a report section with:

- The scaling strategy you would use if your application had 10x, 100x, 1000x more users
- The cost implications of each scaling strategy (include sample calculations)
- The tradeoffs you would make in terms of performance, cost and maintainability
- A new architecture diagram showing your scaling strategy

- Introduce in-memory caching (Redis/ElastiCache) for high-frequency read requests to reduce database load.
- Implement CDN caching (CloudFront) for static assets and API caching for read endpoints.

Expected Cost Implications:

- Fargate (average scaled): assume average 20 mid-sized tasks × \$50
= \$1,000
- ALB & heavier traffic costs: \$50
- CloudFront: \$50
- ElastiCache (Redis): \$120
- Total: \$1300 increment

Tradeoffs:

- Performance: Much better concurrency and resilience. Autoscaling handles bursts.
- Cost: Higher ongoing cost compared to vertical scaling, but more cost-effective per request at this scale because you only run many tasks when needed (if autoscaling is tuned).
- Maintainability: More components — caches, queues, replicas — increases operational complexity and observability needs (health checks, metrics, tracing).

1000x Scaling Strategy

At 1000x scale of the current load, bottlenecks shift from single components to inter-service interactions, database write throughput, and network bandwidth. The system is now in a high-scale distributed system domain. At this scale following should be our scaling strategy:

- Multi-region deployment for: Latency reduction, Fault isolation, Availability zone/regional failover
- Add a dedicated read replica for RDS to distribute read-heavy workloads.
- Move compute-intensive processing tasks to serverless or batch frameworks
- Advanced autoscaling policies: Predictive autoscaling using historical demand patterns

Expected Cost Implications:

- RDS primary + read replica: \$150 (primary upgraded) + \$150 (replica) = \$300
- Replicating architecture across AZs and coordinating between them: +\$3000 per AZ
- CloudFront: \$300
- ALBs / networking: \$200
- Total: \$15000 - 20000 per month increment

Tradeoffs:

- Performance: High throughput and low latency globally if properly provisioned; multi-region reduces user-perceived latency.
- Cost: Very high; must be optimized (reserved instances, savings plans, capacity planning, CDNs). A study is required to best optimize cost and performance.

- Maintainability: Substantial complexity increase, more operational burden (cross-region DR, data replication, partitioning logic).

4 Submission Instructions

Submit a pull request to the course Github repo with your assignment in a folder named a5 with your a5.pdf on a branch called a5, include your team members names on the first page with student IDs. Also upload your assignment to quercus.